

# The Mathematical Expression of Convolutional Neural Networks with the MNIST Dataset

Tsukasa Miyaji, Yusei Fujikura

June 5, 2024

## **Abstract**

This paper presents a detailed mathematical expression of Convolutional Neural Networks (CNNs) using the MNIST dataset. We explore the fundamental concepts and architectures that make CNNs a powerful tool for image recognition tasks. Our discussion includes an overview of the convolutional layers, pooling layers, and the backpropagation algorithm tailored for CNNs. The MNIST dataset serves as an exemplary benchmark to illustrate these concepts in practice.

# Convolutional Neural Network

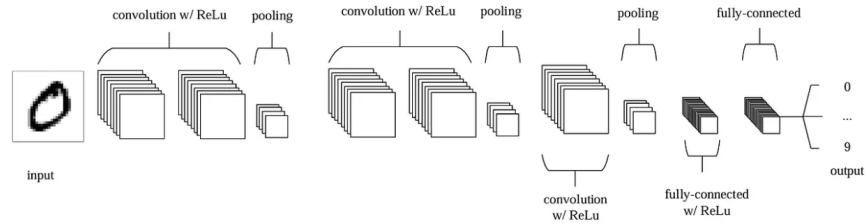


Figure 1: Convolutional Neural Network Architecture

## Forward Propagation in Convolutional Neural Network

In a Convolutional Neural Network, we can split the architecture into three different parts:

1. Convolutional layer with ReLU (or any other activation function).
2. Pooling layer.
3. Fully-connected layer.

## Convolutional Layer

In the first part, we use input images and kernels (also called filters) to automatically and efficiently extract spatial features like edges, textures, and shapes from images. Below is an illustration of the flow in the convolutional layer with the input image and kernel matrix.

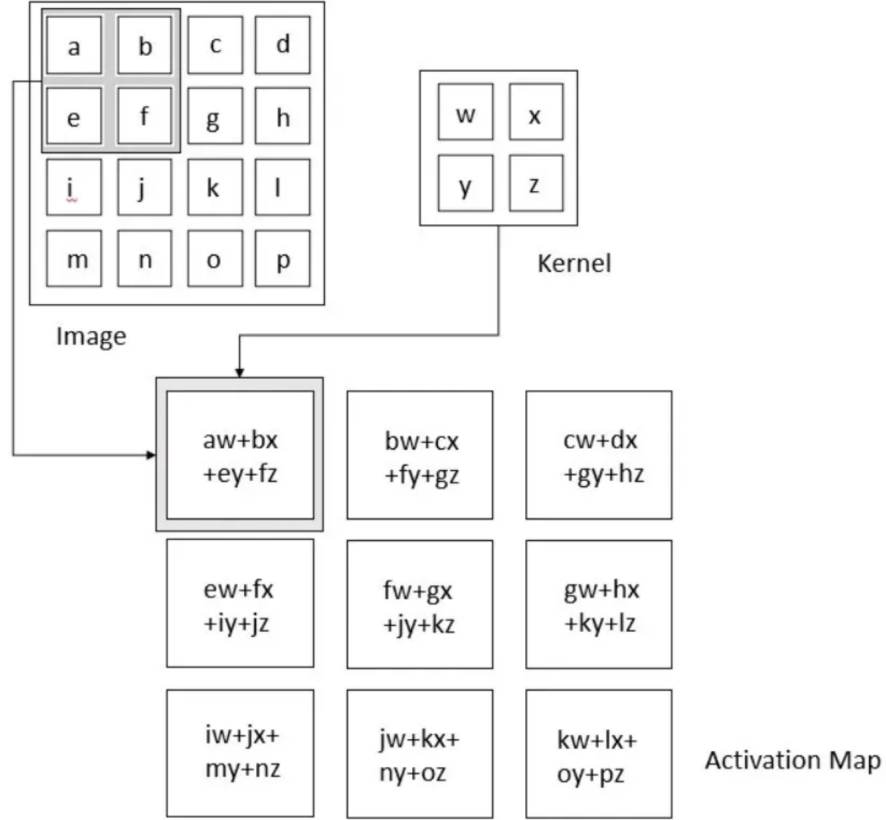


Figure 2: Convolution Operation Example

The convolution operation can be mathematically expressed as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Where:

- $S(i, j)$  is the output feature map.
- $I$  is the input image.
- $K$  is the kernel or filter.
- $i, j$  are the coordinates on the feature map.
- $m, n$  are the coordinates in the kernel.
- $*$  denotes the convolution operation (element-wise matrix multiplication).

From the convolution operation, we obtain the output feature map. This new matrix represents the presence and intensity of features detected by the kernel at specific locations in the input image. We also need to consider padding and stride to fully understand the mechanism of CNNs.

## Stride

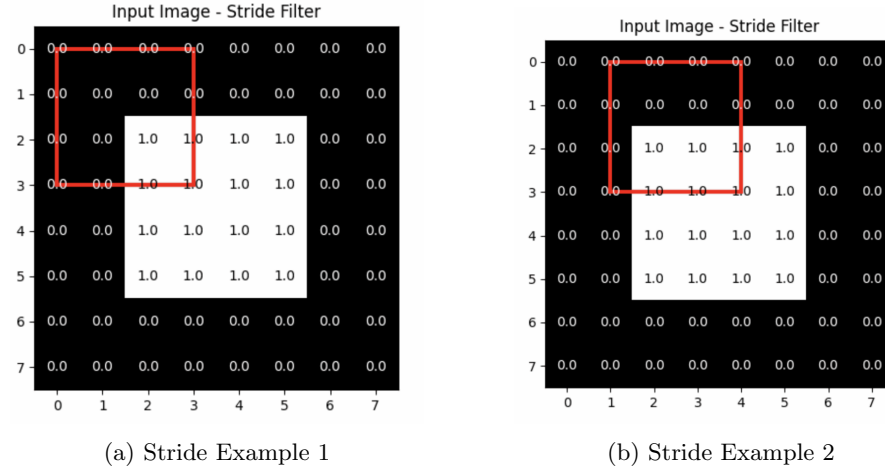


Figure 3: Stride Examples

Stride is a crucial concept in the architecture of CNNs, particularly within the convolutional layers. It fundamentally influences how the kernel traverses the input image or feature map.

The stride specifies the number of pixels by which we move the filter across the input image or feature map in each step. It is applied both horizontally and vertically. A stride of 1 means the filter moves one pixel at a time, ensuring detailed and dense scanning of the input. Larger strides result in the filter skipping pixels, leading to broader and less dense coverage. In our model, we set the stride to 1.

## Padding

In the above image, there are 0.0 elements around the actual picture input. This is called padding. In the above cases, the padding is equal to 2. Without padding, each convolution operation reduces the size of the feature map. Padding allows us to apply filters to the input without shrinking its spatial dimensions, preserving more information, especially for deeper networks where many convolutional layers are applied sequentially.

By padding the input, filters can properly process the edge pixels of the image, ensuring that features located at the borders are adequately captured and utilized in the network’s learning process. In our model, we do not set any padding, called valid padding.

The effect of padding on the output feature map size can be captured by adjusting the formula used to calculate the dimensions of the output feature map:

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

Adjustment of Feature Map Formula with Padding

Where:

- $W_{\text{out}}$  and  $H_{\text{out}}$  are the width and height of the output feature map, respectively.
- $W_{\text{in}}$  and  $H_{\text{in}}$  are the width and height of the input, respectively.
- $F$  is the size of the filter/kernel.
- $S$  is the stride.
- $P$  is the amount of padding added to each side of the input.

## Pooling

Pooling layers serve to reduce the spatial dimensions of the feature maps. This reduction is crucial for decreasing the computational load, minimizing overfitting, and retaining only the most essential information. Let’s delve into the specifics of pooling layers, their types, and their impact on CNN performance.

Pooling layers reduce the size of the feature maps, thereby decreasing the number of parameters and computations required in the network. This simplification helps to focus on the most important features.

By summarizing the presence of features in patches of the feature map, pooling helps the network maintain robustness to minor variations and translations in the input image.

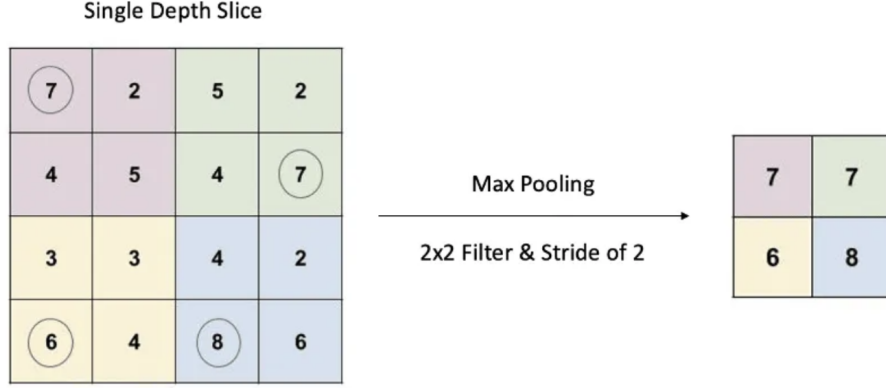


Figure 4: Pooling Operation Example

We used the max pooling method to extract specific features of the image and reduce the size, but there are also other methods such as average pooling (taking the average of each small matrix in the feature map) and more.

We denote the feature map by  $F$  and the pooling operation by  $P_{\max}$ . The result of max pooling at position  $(i, j)$  for a window size of  $n \times n$  can be expressed as:

$$P_{\max}(i, j) = \max_{a=0}^{n-1} \max_{b=0}^{n-1} F(i \cdot s + a, j \cdot s + b)$$

Max Pooling Formula

Here,  $s$  is the stride of the pooling window, and  $a, b$  iterate over the window dimensions. This operation is applied independently for each window position across the feature map.

We can also write down the formula for average pooling:

For a feature map  $F$  and an  $n \times n$  pooling window, the average pooling operation at position  $(i, j)$  can be mathematically represented as:

$$P_{\text{avg}}(i, j) = \frac{1}{n^2} \sum_{a=0}^{n-1} \sum_{b=0}^{n-1} F(i \cdot s + a, j \cdot s + b)$$

Average Pooling Formula

Similar to max pooling,  $s$  represents the stride, and  $a, b$  iterate over the window, but here the operation computes the mean of the values within each window.

## Flatten Layer

Before moving to the fully-connected layer, we need to change the matrix shape (after passing through convolutional and pooling layers, the shape is still a

matrix) into a vector to use as input for the fully-connected layer.

The flatten layer in a Convolutional Neural Network (CNN) is used to transform the multidimensional output of the convolutional and pooling layers into a one-dimensional vector, which can then be fed into fully connected layers for classification or regression tasks.

If the output from the previous layer is a matrix with dimensions  $(d_1, d_2)$  (height, width), the flatten layer transforms this matrix into a vector of length  $d_1 \times d_2$ .

For example, if the input tensor  $\mathbf{I}$  has dimensions  $(2, 3)$ :

$$\mathbf{I} = \begin{bmatrix} i_{111} & i_{112} & i_{113} \\ i_{121} & i_{122} & i_{123} \end{bmatrix}$$

The flatten layer converts this into a vector:

$$\mathbf{v} = [i_{111}, i_{112}, i_{113}, i_{121}, i_{122}, i_{123}]$$

## Fully-Connected Layer

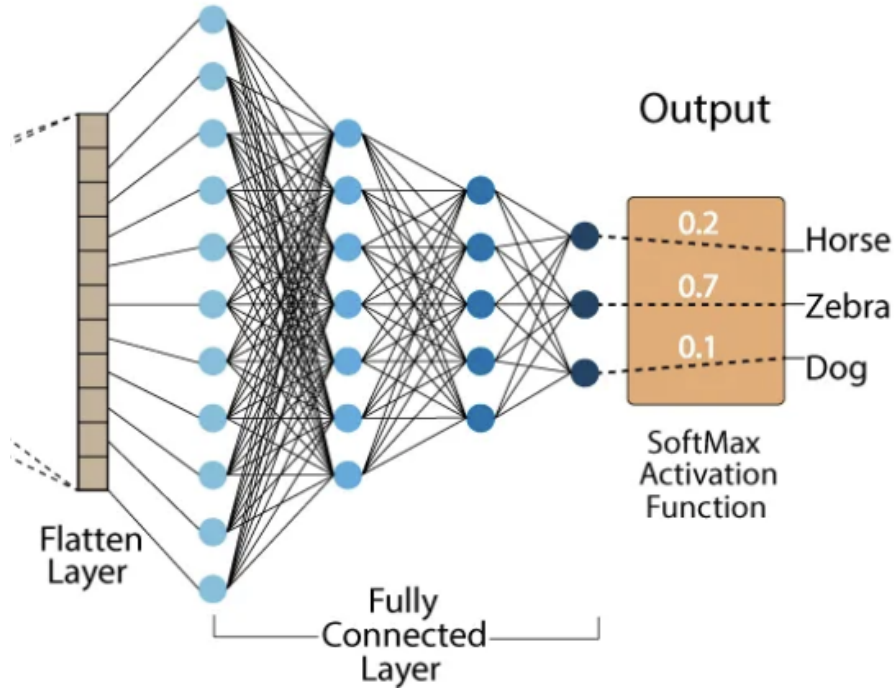


Figure 5: Fully-Connected Layer Example

Fully connected layers are often positioned towards the end of CNNs. These layers are where the high-level reasoning based on the learned features takes place, ultimately leading to classification or prediction. In a fully connected layer, every neuron is connected to every activation from the previous layer. This dense connectivity ensures that the layer has the complete context of the extracted features, allowing it to learn complex patterns that are distributed across the feature map.

Fully connected layers integrate the spatially distributed features identified by convolutional and pooling layers into a global representation of the input. This integration is crucial for tasks that require an understanding of the entire input, such as classification.

The output of a fully connected layer can be represented as follows:

$$y_i = f \left( \sum_{j=1}^N w_{ij} x_j + b_i \right)$$

Where:

- $y_i$  is the output of the  $i$ -th neuron in the fully connected layer.
- $f$  is an activation function (such as ReLU, sigmoid, or softmax; we used ReLU and softmax).
- $w_{ij}$  is the weight connecting the  $j$ -th neuron of the previous layer to the  $i$ -th neuron of the fully connected layer.
- $x_j$  is the input from the  $j$ -th neuron of the previous layer.
- $b_i$  is the bias term for the  $i$ -th neuron.
- $N$  is the number of inputs to the fully connected layer.

In this model, the ReLU function is used for intermediate activations to add non-linearity, while the Softmax function is used at the end to produce the final class probabilities.



## 1 Introduction

Backpropagation is a crucial algorithm for training neural networks, including Convolutional Neural Networks (CNNs). It computes the gradient of the loss function with respect to each weight by the chain rule, enabling the optimization of the network's parameters. This document provides a mathematical breakdown of backpropagation in CNNs.

## 2 Notation

- $L$ : Loss function
- $W^{(l)}$ : Weights at layer  $l$
- $b^{(l)}$ : Bias at layer  $l$
- $a^{(l)}$ : Activations at layer  $l$
- $z^{(l)}$ : Weighted sum at layer  $l$ ,  $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$
- $\sigma$ : Activation function

## 3 Forward Pass

The forward pass computes the activations of each layer given the input  $x$ :

$$a^{(0)} = x \tag{1}$$

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \tag{2}$$

$$a^{(l)} = \sigma(z^{(l)}) \tag{3}$$

## 4 Loss Function

Assume a loss function  $L$  (e.g., mean squared error, cross-entropy):

$$L = \text{Loss}(a^{(L)}, y) \tag{4}$$

where  $a^{(L)}$  is the output of the network and  $y$  is the true label.

## 5 Backward Pass

The backward pass computes the gradients of the loss with respect to the weights and biases by propagating the error backward through the network.

### 5.1 Output Layer

For the output layer  $L$ , the error  $\delta^{(L)}$  is:

$$\delta^{(L)} = \frac{\partial L}{\partial z^{(L)}} = (a^{(L)} - y) \odot \sigma'(z^{(L)}) \quad (5)$$

### 5.2 Hidden Layers

For hidden layers, the error  $\delta^{(l)}$  is:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)}) \quad (6)$$

### 5.3 Gradients

The gradients of the loss with respect to the weights and biases are:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T \quad (7)$$

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)} \quad (8)$$

## 6 Convolutional Layers

In CNNs, the forward and backward passes through convolutional layers involve specific operations.

### 6.1 Forward Pass in Convolutional Layers

For a convolutional layer with input  $a^{(l-1)}$  and filter  $W^{(l)}$ :

$$z^{(l)}(i, j) = \sum_m \sum_n W^{(l)}(m, n) a^{(l-1)}(i - m, j - n) + b^{(l)} \quad (9)$$

### 6.2 Backward Pass in Convolutional Layers

The error for the convolutional layer is:

$$\delta^{(l)}(i, j) = \sum_{p, q} (W^{(l+1)}(p, q) \delta^{(l+1)}(i + p, j + q)) \odot \sigma'(z^{(l)}(i, j)) \quad (10)$$

The gradients are:

$$\frac{\partial L}{\partial W^{(l)}}(m, n) = \sum_{i, j} \delta^{(l)}(i, j) a^{(l-1)}(i - m, j - n) \quad (11)$$

$$\frac{\partial L}{\partial b^{(l)}} = \sum_{i, j} \delta^{(l)}(i, j) \quad (12)$$

## 7 Pooling Layers

For pooling layers, the error is propagated to the corresponding locations of the maximum values in the input.

### 7.1 Max Pooling

If  $P^{(l)}$  is the max-pooled output, the gradient with respect to the input  $a^{(l)}$  is:

$$\delta^{(l)}(i, j) = \begin{cases} \delta^{(l+1)}(i', j') & \text{if } a^{(l)}(i, j) = P^{(l)}(i', j') \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

## 8 Update Rule

Finally, the weights and biases are updated using gradient descent:

$$W^{(l)} := W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} \quad (14)$$

$$b^{(l)} := b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}} \quad (15)$$

where  $\eta$  is the learning rate.

## 9 Conclusion

Backpropagation in CNNs involves computing the gradients of the loss function with respect to each layer's weights and biases by propagating errors backward through the network. This process is essential for optimizing the network's parameters and improving its performance on the given task.

# MATH156\_CNN\_final\_project

June 7, 2024

## 1 Import required packages

```
[10]: # Import Tensorflow
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
from sklearn.preprocessing import LabelEncoder
from sklearn import preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import seaborn as sns
import tensorflow as tf
from pympler import asizeof
import tensorflow_datasets as tfds
import psutil
from tensorflow import keras
np.random.seed(0)
```

## 2 CNN

```
[11]: # Random seed for reproducibility
seed = 42
tf.keras.utils.set_random_seed(seed)

# Save the model at the end?
save_model = False

# Batch sizes for training and testing
batch_size = 64
test_batch_size = 14

# Training epochs
n_epochs = 5

# Learning rate
```

```

learning_rate = 1.0

# Decay rate for adjusting the learning rate
gamma = 0.7

# Number of target classes in the MNIST data
num_classes = 10

# Data input shape
input_shape = (28, 28, 1)

```

```

[12]: # Load the MNIST dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# The scaled mean and standard deviation of the MNIST dataset (precalculated)
data_mean = 0.1307
data_std = 0.3081

# Reshape the input data
x_train = x_train.reshape(x_train.shape[0], x_train.shape[1], x_train.shape[2], 1)

x_test = x_test.reshape(x_test.shape[0],
                        x_test.shape[1],
                        x_test.shape[2], 1)

# Normalize the data
x_train = (x_train/255.0 - data_mean) / data_std
x_test = (x_test/255.0 - data_mean) / data_std

# Convert labels to one-hot vectors
y_train = tf.one_hot(y_train.astype(np.int32), depth=num_classes)
y_test = tf.one_hot(y_test.astype(np.int32), depth=num_classes)

```

```

[13]: # Define the architecture of the neural network
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), strides=(1,1),
                           padding='valid',
                           activation='relu',
                           input_shape=input_shape),
    tf.keras.layers.Conv2D(64, (3,3), strides=(1,1),
                           padding='valid',
                           activation='relu'),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Flatten(),

```

```
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dropout(0.5),
tf.keras.layers.Dense(num_classes, activation='softmax'))])
```

```
[15]: # Decay the learning rate at a base rate of gamma roughly every epoch, which
# is len(x_train) steps
scheduler = tf.keras.optimizers.schedules.ExponentialDecay(
    learning_rate,
    decay_steps=len(x_train),
    decay_rate=gamma)

# Define the optimizer to use for gradient descent
optimizer = tf.keras.optimizers.legacy.Adadelta(scheduler)

# Compile the model
model.compile(optimizer=optimizer, loss='categorical_crossentropy',
    metrics=['acc'])

# Display a model summary
model.summary()
```

Model: "sequential\_1"

| Layer (type)                        | Output Shape       | Param # |
|-------------------------------------|--------------------|---------|
| conv2d_2 (Conv2D)                   | (None, 26, 26, 32) | 320     |
| conv2d_3 (Conv2D)                   | (None, 24, 24, 64) | 18496   |
| max_pooling2d_1 (MaxPooling2D)      | (None, 12, 12, 64) | 0       |
| dropout_2 (Dropout)                 | (None, 12, 12, 64) | 0       |
| flatten_1 (Flatten)                 | (None, 9216)       | 0       |
| dense_2 (Dense)                     | (None, 128)        | 1179776 |
| dropout_3 (Dropout)                 | (None, 128)        | 0       |
| dense_3 (Dense)                     | (None, 10)         | 1290    |
| Total params: 1199882 (4.58 MB)     |                    |         |
| Trainable params: 1199882 (4.58 MB) |                    |         |
| Non-trainable params: 0 (0.00 Byte) |                    |         |

```
[16]: def get_cpu_usage():
        return psutil.cpu_percent(interval=1)

model.fit(x_train, y_train,
          batch_size=batch_size,
          epochs=n_epochs,
          validation_data=(x_test, y_test),
          validation_batch_size=test_batch_size)
```

```
Epoch 1/5
938/938 [=====] - 38s 40ms/step - loss: 0.1826 - acc:
0.9447 - val_loss: 0.0469 - val_acc: 0.9846
Epoch 2/5
938/938 [=====] - 37s 39ms/step - loss: 0.0739 - acc:
0.9785 - val_loss: 0.0353 - val_acc: 0.9881
Epoch 3/5
938/938 [=====] - 37s 39ms/step - loss: 0.0546 - acc:
0.9837 - val_loss: 0.0368 - val_acc: 0.9883
Epoch 4/5
938/938 [=====] - 37s 40ms/step - loss: 0.0460 - acc:
0.9863 - val_loss: 0.0323 - val_acc: 0.9895
Epoch 5/5
938/938 [=====] - 37s 39ms/step - loss: 0.0402 - acc:
0.9878 - val_loss: 0.0292 - val_acc: 0.9912
```

```
[16]: <keras.src.callbacks.History at 0x2a6631750>
```

## References

- [1] "Understanding Convolutional Neural Networks: A Beginner's Journey into the Architecture," *Medium*, 2019. [Online]. Available: <https://medium.com/codex/understanding-convolutional-neural-networks-a-beginners-journey-into-the-architecture-aab30dface10> [Accessed: Jun. 7, 2024].
- [2] "Convolution, Padding, Stride and Pooling in CNN," *Medium*, 2019. [Online]. Available: <https://medium.com/analytics-vidhya/convolution-padding-stride-and-pooling-in-cnn-13dc1f3ada26>
- [3] "What is Stride in Convolutional Neural Network," *Medium*, 2019. [Online]. Available: <https://medium.com/machine-learning-algorithms/what-is-stride-in-convolutional-neural-network-e3b4ae9baedb>
- [4] "The Math Behind Convolutional Neural Networks," *Towards Data Science*, 2019. [Online]. Available: <https://towardsdatascience.com/the-math-behind-convolutional-neural-networks-6aed775df076>.
- [5] "Forward and Backward Propagation in Convolutional Neural Networks," *Towards Data Science*, 2019. [Online]. Available: <https://towardsdatascience.com/forward-and-backward-propagation-in-convolutional-neural-networks-64365925fdfa>
- [6] "CNN Backpropagation Between Layers," *Data Science Stack Exchange*, 2019. [Online]. Available: <https://datascience.stackexchange.com/questions/41569/cnn-backpropagation-between-layers/41584#41584>
- [7] Pavisj, "Convolutions and Backpropagations," *Medium*, 2019. [Online]. Available: <https://pavisj.medium.com/convolutions-and-backpropagations-46026a8f5d2c>

## Repository

The code and additional resources for this paper can be found at the following repository: [https://github.com/TsukasaMiyaji/MATH156\\_final\\_project.git](https://github.com/TsukasaMiyaji/MATH156_final_project.git)